

You are here: [Home](#) » [Cisco](#) » [CCNA 200-301](#)

TCP Window Size Scaling

Lesson Contents

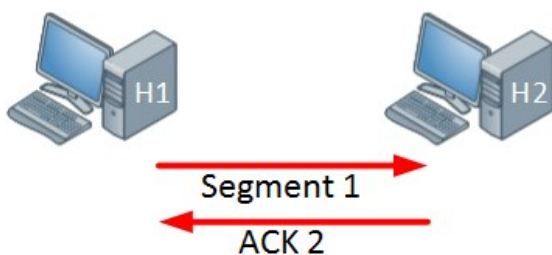
1. [Wireshark Captures](#)
2. [Conclusion](#)

TCP (Transmission Control Protocol) is a connection oriented protocol which means that we keep track of how much data has been transmitted. The sender will transmit some data and the receiver has to acknowledge it. When we don't receive the acknowledgment in time then the sender will re-transmit the data.

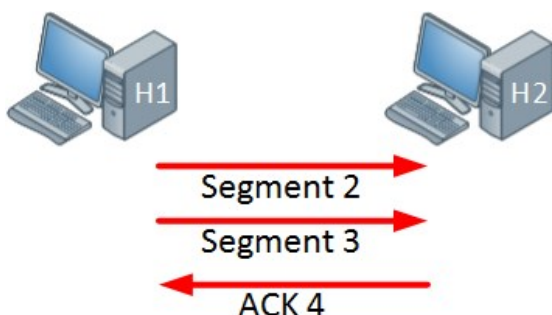
TCP uses "windowing" which means that a sender will send one or more data segments and the receiver will acknowledge one or all segments. When we start a TCP connection, the hosts will use a receive buffer where we temporarily store data before the application can process it.

When the receiver sends an acknowledgment, it will tell the sender how much data it can transmit before the receiver will send an acknowledgment. We call this the **window size**. Basically, the window size indicates the size of the receive buffer.

Typically the TCP connection will start with a small window size and every time when there is a successful acknowledgement, the window size will increase. Here's an example:



Above we have two hosts, the host on the left side will send one segment and the host on the right side will send an acknowledgment in return. Since the acknowledgement was successful, the windows size will increase:

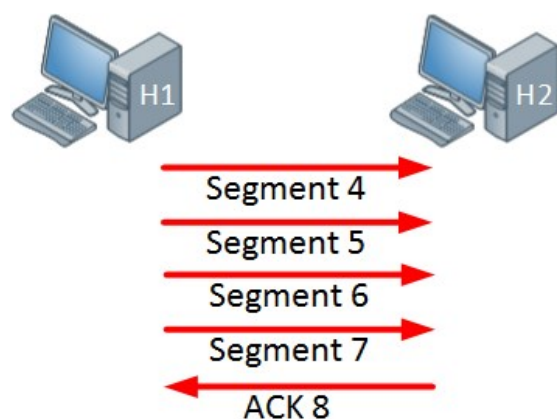


Course Contents

CCNA 200-301

- + Unit 1: Introduction
- Unit 2: Network Fundamentals
 - [Introduction to the OSI Model](#)
 - [Introduction to IPv4 \(Internet Protocol\)](#)
 - [IPv4 Packet Header](#)
 - [Address Resolution Protocol \(ARP\)](#)
 - [Introduction to TCP and UDP](#)
 - [TCP Header](#)
 - [TCP Window Size Scaling](#)**
 - [Introduction to ICMP](#)
 - [Introduction to DNS](#)
 - [Introduction to Cisco Command Line Interface \(CLI\)](#)
 - [User mode and Privileged mode](#)
- + Unit 3: Network Access
- + Unit 4: IP Connectivity
- + Unit 5: IP Services
- + Unit 6: IPv6
- + Unit 7: Security Fundamentals
- + Unit 8: Network Management
- + Unit 9: Network Design
- + Unit 10: Automation and Programmability
- + Unit 11: Cloud Computing
- + Unit 12: Practice Exam

The host on the left side is now sending two segments and the host on the right side will return a single acknowledgment. Everything is working fine so the window size will increase even further:



The host is now sending four segments and the host on the right side responds with a single acknowledgment.

In the example above the window size keeps increasing as long as the receiver sends acknowledgments for all our segments or when the window size hits a certain maximum limit. When the receiver doesn't send an acknowledgment within a certain time period (called the round-trip time) then the window size will be reduced.

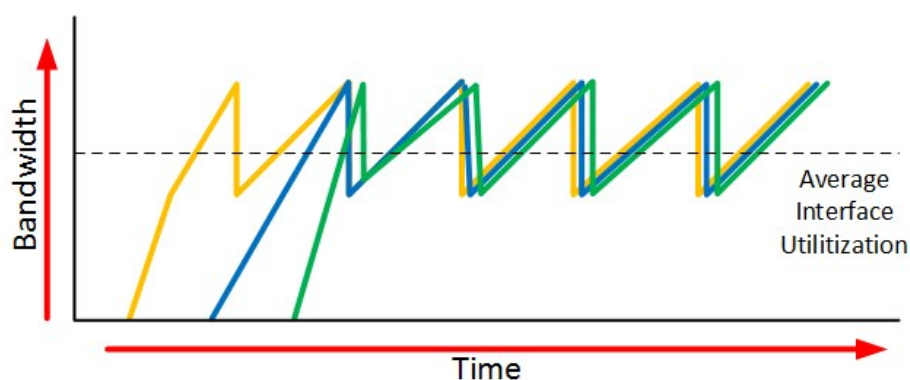
When an interface has congestion then it's possible that IP packets are dropped. To deal with this, TCP has a number of algorithms that deal with congestion control. One of them is called *slow start*.



Congestion occurs when the interface has to transmit more data than it can handle. It's queue(s) will hit a limit and packets will be dropped.

With TCP slow start, the window size will initially grow exponentially (window size doubles) but once a packet is dropped, the window size will be reduced to one segment. It will then grow exponentially again until the window size is half of what it was when the congestion occurred. At that moment, the window size will grow linearly instead of exponentially.

When an interface gets congested, it's possible that all your TCP connections will experience TCP slow start. Packets will be dropped and then all TCP connections will have a small window size. This is called **TCP global synchronization**. Here's what it looks like:



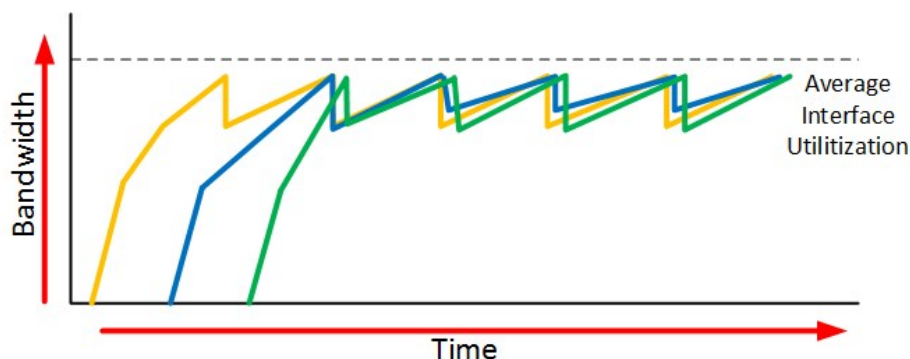
The orange, blue and green lines are three different TCP connections. These TCP connections start at different times and after awhile, the interface gets congested and packets of all TCP connections are dropped. What happens is that the window size of

all these TCP connections will drop to one and once the interface congestion is gone, all their window sizes will increase again.

The interface then gets congested again, the window size drops back to one and the story repeats itself. The result of this is that we don't use all the available bandwidth that our interface has to offer. If you look at the dashed line you can see that we the average interface utilization isn't very high.

To prevent global synchronization we can use [RED \(Random Early Detection\)](#). this is a feature that drops "random" packets from TCP flows based on the number of packets in a queue and the TOS (Type of Service) marking of the packets. When packets are dropped before a queue is full, we can avoid the global synchronization.

The end result will look similar to this:



When we use RED, our average interface utilization will improve.

Now you have an idea what the TCP window size is about, let's take a look at a real example of how the window size is used. We can use wireshark for this.

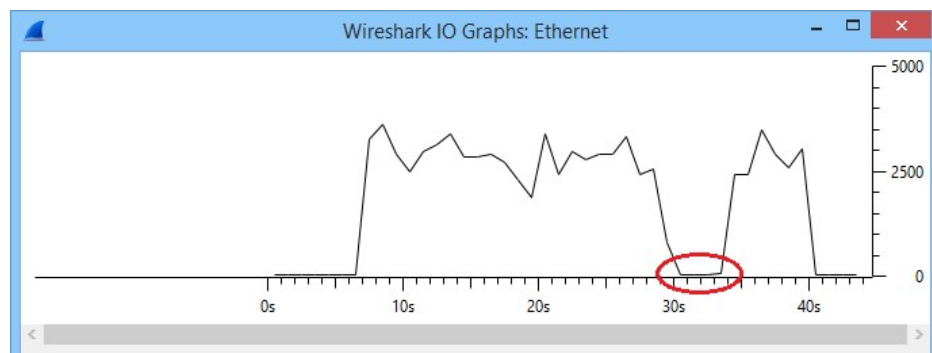
1. Wireshark Captures

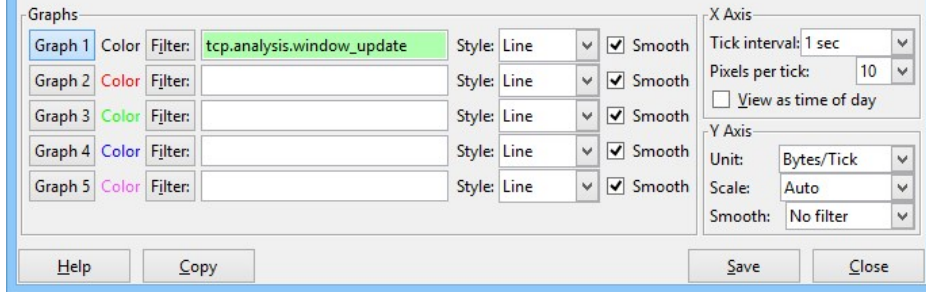
To examine the TCP window size I will use two devices:



The device on the left side is a modern computer with a gigabit interface. On the right side, we have a small raspberry pi which has a FastEthernet interface. The raspberry pi is a great little device but it's cpu / memory / ethernet interface are limited. To get an interesting output, I will copy a large file through SSH from my computer to the raspberry pi which will be easily overburdened.

Here's what happened, take a look at this picture:





In the graph above you can see the window size that was used during this connection. The file transfer started after about 6 seconds and you can see that the window size increased fast. It went up and down a bit but at around 30 seconds, it totally collapsed. After a few seconds it increased again and I was able to complete the file transfer. Let's take a closer look at this file transfer, which starts with the three way handshake:

No.	Time	Source	Destination	Protocol	Length	Info
475	6.531678000	10.56.100.1	10.56.100.164	TCP	66	56748->22 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=128 SACK_PERM=1
476	6.532095000	10.56.100.164	10.56.100.1	TCP	66	22->56748 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM=1 WS=64
477	6.532144000	10.56.100.1	10.56.100.164	TCP	54	56748->22 [ACK] Seq=1 Ack=1 Win=4194304 Len=0

My fast computer uses 10.56.100.1 and the raspberry pi uses 10.56.100.164. Above you can see that in the SYN,ACK message that the raspberry pi wants to use a window size of 29200. My computer wants to use a window size of 8388480 (win=65535 * ws=128) which is irrelevant now since we are sending data to the raspberry pi.

After a few packets, the window size of the raspberry pi looks like this:

```

Frame 639: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0
Ethernet II, Src: Raspberr_68:1e:36 (b8:27:eb:68:1e:36), Dst: AsustekC_7d:22:8c (74:d0:2b:7d:22:8c)
Internet Protocol Version 4, Src: 10.56.100.164 (10.56.100.164), Dst: 10.56.100.1 (10.56.100.1)
Transmission Control Protocol, Src Port: 22 (22), Dst Port: 56748 (56748), Seq: 2520, Ack: 51956, Len: 0
  Source Port: 22 (22)
  Destination Port: 56748 (56748)
  [Stream index: 16]
  [TCP Segment Len: 0]
  Sequence number: 2520 (relative sequence number)
  Acknowledgment number: 51956 (relative ack number)
  Header Length: 20 bytes
  ... 0000 0001 0000 = Flags: 0x010 (ACK)
    ...0 ... = Reserved: Not set
    ...0 ... = Nonce: Not set
    ...0 ... = Congestion Window Reduced (CWR): Not set
    ...0 ... = ECN-Echo: Not set
    ...0 ... = Urgent: Not set
    ...1 ... = Acknowledgment: Set
    ...0 ... = Push: Not set
    ...0 ... = Reset: Not set
    ...0 ... = Syn: Not set
    ...0 ... = Fin: Not set
  window size value: 2070
  [Calculated window size: 132480]
  [window size scaling factor: 64]
  Checksum: 0x102f [validation disabled]
  urgent pointer: 0
  [SEQ/ACK analysis]

```

Above you can see that the window size has increased to 132480. Originally the window size is a 16 bit value so the largest window size would be 65535. Nowadays we use a scaling factor so that we can use larger window sizes.

At around the 10 second mark the window size decreased. Here's what happened:

6455	9.587872000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6456	9.597749000	10.56.100.164	10.56.100.1	TCP	60	22->56748 [ACK] Seq=34920 Ack=7228667 Win=26752 Len=0
6457	9.597847000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6458	9.597860000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6459	9.597871000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6460	9.597882000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6461	9.597892000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6462	9.597903000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6463	9.597914000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6464	9.597925000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6465	9.597936000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6466	9.597946000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6467	9.598024000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6468	9.598037000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6469	9.598049000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6470	9.598060000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6471	9.598071000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6472	9.598082000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6473	9.598094000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6474	9.598105000	10.56.100.1	10.56.100.164	SSHv2	1514	Client: Encrypted packet (len=1460)
6475	9.598116000	10.56.100.1	10.56.100.164	SSHv2	526	Client: [TCP Window Full], Encrypted packet (len=472)

The raspberry pi seems to have trouble keeping up and its receive buffer is probably full. It tells the computer to use a window size of 26752 from now on. The computer sends 18 segments with 1460 bytes and one segment of 472 bytes (26752 bytes in total). The last packet shows us "TCP Window Full" message. This is something that wireshark reports to us, our computer has completely filled the receive buffer of the

raspberry pi.

Once the raspberry pi has caught up a bit and around the 30 second mark, something bad happens. Take a look at the wireshark capture below:

53980	34.339849000	10.56.100.1	10.56.100.164	SSHv2	910 Client: [TCP window Full] , Encrypted packet (len=856)
53981	34.339974000	10.56.100.164	10.56.100.1	SSHv2	166 Server: Encrypted packet (len=112)
53982	34.343453000	10.56.100.164	10.56.100.1	TCP	60 [TCP zerowindow] 22-56748 [ACK] Seq=268280 Ack=66840816 win=0 Len=0

Above you can see that the raspberry pi sends an ACK to the computer with a **window size of 0**. This means that the window size will remain at 0 for a specified amount of time, the raspberry pi is unable to receive any more data at this moment and the TCP transmission will be paused for awhile while the receive buffer is processed.

Here's the actual packet:

Frame 53982: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0	
Ethernet II, Src: Raspberr_68:1e:36 (b8:27:eb:68:1e:36), Dst: AsustekC_7d:22:8c (74:d0:2b:7d:22:8c)	
Internet Protocol Version 4, Src: 10.56.100.164 (10.56.100.164), Dst: 10.56.100.1 (10.56.100.1)	
Transmission Control Protocol, Src Port: 22 (22), Dst Port: 56748 (56748), Seq: 268280, Ack: 66840816, Len: 0	
Source Port: 22 (22)	
Destination Port: 56748 (56748)	
[Stream index: 16]	
[TCP Segment Len: 0]	
Sequence number: 268280 (relative sequence number)	
Acknowledgment number: 66840816 (relative ack number)	
Header Length: 20 bytes	
0000 0001 0000 = Flags: 0x010 (ACK)	
000. = Reserved: Not set	
...0 = Nonce: Not set	
...0... = Congestion window Reduced (CWR): Not set	
...0.. = ECN-Echo: Not set	
...0. = Urgent: Not set	
...1... = Acknowledgment: Set	
...0... = Push: Not set	
...0.. = Reset: Not set	
...0. = Syn: Not set	
...0 = Fin: Not set	
window size value: 0	
[calculated window size: 0]	
[window size scaling factor: 64]	
Checksum: 0xe829 [validation disabled]	
Urgent pointer: 0	
[SEQ/ACK analysis]	

Above you can see that the window size is now 0. Once the receive buffer has been processed, the raspberry pi will send an ACK with a new window size:

Frame 3659: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0	
Ethernet II, Src: Raspberr_68:1e:36 (b8:27:eb:68:1e:36), Dst: AsustekC_7d:22:8c (74:d0:2b:7d:22:8c)	
Internet Protocol Version 4, Src: 10.56.100.164 (10.56.100.164), Dst: 10.56.100.1 (10.56.100.1)	
Transmission Control Protocol, Src Port: 22 (22), Dst Port: 56748 (56748), Seq: 23672, Ack: 4472703, Len: 0	
Source Port: 22 (22)	
Destination Port: 56748 (56748)	
[Stream index: 0]	
[TCP Segment Len: 0]	
Sequence number: 23672 (relative sequence number)	
Acknowledgment number: 4472703 (relative ack number)	
Header Length: 20 bytes	
0000 0001 0000 = Flags: 0x010 (ACK)	
000. = Reserved: Not set	
...0 = Nonce: Not set	
...0... = Congestion window Reduced (CWR): Not set	
...0.. = ECN-Echo: Not set	
...0. = Urgent: Not set	
...1... = Acknowledgment: Set	
...0... = Push: Not set	
...0.. = Reset: Not set	
...0. = Syn: Not set	
...0 = Fin: Not set	
window size value: 400	
[calculated window size: 25600]	
[window size scaling factor: 64]	
Checksum: 0x4f46 [validation disabled]	
Urgent pointer: 0	
[SEQ/ACK analysis]	

The window size is now only 25600 bytes but will grow again. The rest of the transmission went without any hiccups and the file transfer completed.

2. Conclusion

You have now seen how TCP uses the window size to tell the sender how much data to transmit before it will receive an acknowledgment. I also showed you an example of how the window size is used when the receiver is unable to process its receive buffer in time.



UDP, unlike TCP is a connectionless protocol and will just keep sending traffic. There is no window size, for this reason you might want to limit your UDP traffic or you might see starvation of your TCP traffic when there is

congestion.

I hope you have enjoyed this lesson, if you have any more questions feel free to leave a comment in our forum.

« **Previous Lesson**
TCP Header

Next Lesson »
Introduction to ICMP

Tags: [IP Routing](#)

Forum Replies



michmoor

Bravo. Even with me level of experience you cleared up some concepts for me. I love this site! Thanks Rene



ReneMolenaar

Thanks Michael!



chrismchavez

Rene,

Thanks, that was nice to see this in detail like you showed.

Chris



srahimkhani

Dear Rene,

Thanks, you explain every thing very nice. May I have capture file for this lesson.



ReneMolenaar

Hmm I'm not sure if I still have it, I'll take a look.

💬 **69 more replies!** Ask a question or join the discussion by visiting our [Community Forum](#)

